# Proyecto I — Parte B: pipeline de datos de diabetes

Explicación de la Parte B y de los conceptos nuevos que aparecen, para repasar de cara al examen.

## 1. Qué se ha hecho

**Objetivo:** aplicar la misma arquitectura de la Parte A (cargar → validar → métricas → CLI) a un **dataset público real**, adaptando las reglas y las métricas al dominio clínico. Además, el resultado sirve como primera fase de limpieza del proyecto de Machine Learning.

**Dataset:** *Diabetes 130-US Hospitals for Years 1999-2008* (UCI, licencia CC BY 4.0). Contiene 101 766 **encuentros** (ingresos hospitalarios) de pacientes diabéticos con 50 columnas: datos demográficos, estancia, pruebas, diagnósticos, 23 medicamentos y la variable objetivo `readmitted` (reingreso: `NO`, `>30` días, `<30` días).

**Estructura:**

```
1_programacion_avanzada_python/
  diabetes_app/
    __init__.py
    loader.py          # Protocol EncounterRepository + CsvEncounterRepository +
DataLoadError
    validator.py       # reglas del dominio + ValidationError + motivo de cada fila
inválida
    metrics.py         # 5 métricas + informe de calidad
    cli.py             # python -m diabetes_app.cli --input ... --output ...
    crear_muestra.py   # genera la muestra de 5 000 filas que se sube al repo
    DESIGN.md          # adaptación al dominio, SOLID y decisiones
  Datos/
    diabetic_data.csv          # completo (NO se sube: .gitignore)
    diabetic_data_muestra.csv  # muestra reproducible (SÍ se sube)
  tests/
    test_diabetes.py           # 16 tests (1 de integración)
```

**Ejecución:**

```
python -m diabetes_app.crear_muestra --input Datos/diabetic_data.csv --output Datos/
diabetic_data_muestra.csv
python -m diabetes_app.cli --input Datos/diabetic_data_muestra.csv --output Datos/
diabetes_limpio.csv
```

**Resultados:**

| | Muestra (5 000) | Completo (101 766) |
|---|---|---|
| Válidos | 4 902 | 100 090 |
| Inválidos | 98 | 1 676 |
| Pacientes fallecidos excluidos | 96 | 1 652 |
| Sin diagnóstico principal | 1 | 21 |
| Sexo "Unknown/Invalid" | 1 | 3 |

En el completo: **tasa global de reingreso en menos de 30 días: 11,34 %**. La columna `weight` tiene un **96,9 % de nulos**.

---

## 2. Adaptación al dominio (lo que más se valora)

El enunciado penaliza el "copiar y pegar a ciegas". Estas son las diferencias con la Parte A:

| Aspecto | Parte A (ventas) | Parte B (diabetes) |
|---|---|---|
| Unidad de análisis | Una venta | Un encuentro hospitalario |
| Nulos | Vacíos normales | `?` es nulo; `None` **no** lo es (significa "prueba no realizada") |
| Reglas | 2 columnas > 0 | Nulos, rangos, categorías permitidas, duplicados y fallecidos |
| Filas inválidas | Tal cual | Con columna `motivo` y recuento por regla |
| Métricas | Importe, top productos, clientes recurrentes | Reingresos, estancia, HbA1c, pacientes recurrentes |
| Informe de calidad | Filas e importe | Filas, incumplimientos por regla, % de nulos, columnas casi vacías |

### 2.1 Decisiones de dominio

- **`?` = nulo, `None` ≠ nulo.** pandas convierte por defecto el texto `"None"` en `NaN`. En `A1Cresult` y `max_glu_serum`, `None` significa "no se hizo la prueba", que es información útil (de hecho, una de las métricas compara los pacientes medidos con los no medidos). Por eso el loader usa `keep_default_na=False` y `na_values=["?"]`.
- **Exclusión de pacientes fallecidos** (códigos de alta 11, 19, 20 y 21 de `IDS_mapping.csv`). Un paciente fallecido no puede reingresar, así que incluirlo sesgaría las tasas de reingreso a la baja. Es una regla de **negocio**, no de formato.
- **Columnas casi vacías (`weight` 97 %, `medical_specialty` 49 %, `payer_code` 40 %) no invalidan filas.** Si se exigieran, se perdería casi todo el dataset. Se informan en el JSON de calidad para decidir qué hacer con ellas en el proyecto de ML (normalmente, eliminar `weight`).
- **Varios encuentros por paciente son válidos:** un paciente puede ingresar varias veces. Se informa como métrica.

## 3. Reglas de validación

| # | Regla | Configuración en `validator.py` |
|---|---|---|
| 0 | Columnas obligatorias (si falta alguna → `ValidationError`) | `COLUMNAS_OBLIGATORIAS` |
| 1 | Sin nulos en columnas clave | `COLUMNAS_SIN_NULOS` |
| 2 | Tipo numérico y rango válido | `RANGOS_NUMERICOS = {"time_in_hospital": (1, 14), ...}` |
| 3 | Valores categóricos permitidos | `VALORES_PERMITIDOS = {"gender": {"Female", "Male"}, ...}` |
| 4 | `encounter_id` único | `duplicated(keep="first")` |
| 5 | Pacientes fallecidos excluidos | `CODIGOS_ALTA_FALLECIMIENTO = {11, 19, 20, 21}` |

**Diseño OCP:** las reglas son **datos** (listas y diccionarios), y `construir_reglas()` los recorre en bucles. Para añadir una regla basta con añadir una entrada; la lógica no se toca.

### 3.1 Cómo funciona: un diccionario de máscaras

```
reglas = {
    "gender con valor no permitido": df["gender"].notna() &
~df["gender"].isin({"Female", "Male"}),
    "encounter_id duplicado": df["encounter_id"].duplicated(keep="first"),
    ...
}   # cada máscara vale True en las filas que INCUMPLEN la regla
```

- `construir_reglas()` devuelve `{motivo: máscara}`.
- `validar_encuentros()` asigna a cada fila su **motivo** (la primera regla que incumple). Las filas sin motivo son válidas.
- `contar_incumplimientos()` suma cada máscara (True = 1) para saber cuántas filas incumple cada regla.

## 4. Métricas

| Métrica | Técnica de pandas | Por qué tiene sentido |
|---|---|---|
| Distribución de reingresos (%) | `value_counts(normalize=True) * 100` | Muestra el desbalance de la variable objetivo, clave para el ML |
| Reingreso <30 días por edad | `groupby` + media de un booleano + `reindex` | Detecta los grupos de edad de más riesgo |
| Reingreso <30 días según HbA1c | `groupby("A1Cresult")` | Es la pregunta del artículo original: ¿influye medir la HbA1c? |
| Estancia media por reingreso | `groupby("readmitted")["time_in_hospital"].mean()` | Estancias más largas asociadas a más reingresos |
| Pacientes recurrentes | `value_counts()` + filtro `> 1` | Es la misma idea que los "clientes recurrentes" de la Parte A |

## 5. Conceptos nuevos de pandas

### 5.1 Control de nulos al leer: `na_values` y `keep_default_na`

```
pd.read_csv(ruta, na_values=["?"], keep_default_na=False, low_memory=False)
```

| Parámetro | Qué hace |
|---|---|
| `na_values=["?"]` | Añade `"?"` a los valores que se leen como nulo |
| `keep_default_na=False` | Desactiva la lista por defecto de pandas (`"None"`, `"NA"`, `"null"`, `""`...), que convertiría `"None"` en `NaN` |
| `low_memory=False` | Lee el fichero de una vez, en lugar de por trozos, y evita el aviso `DtypeWarning` de tipos mezclados en una columna |

> **Idea clave:** hay que revisar qué significa cada valor "vacío" en el dominio **antes** de decidir que es nulo.

### 5.2 La media de un booleano es una proporción

```
es_reingreso_30 = validos["readmitted"] == "<30"          # Series de True/False
tasa = es_reingreso_30.groupby(validos["age"]).mean() * 100
```

`True` vale 1 y `False` vale 0, así que la media es la **proporción de True**, y multiplicada por 100 da un porcentaje. Es la forma más directa de calcular tasas por grupo.

### 5.3 `value_counts(normalize=True)`

Devuelve proporciones (que suman 1) en lugar de conteos. Multiplicado por 100 da porcentajes.

### 5.4 `reindex` para ordenar según una lista

```
tasa.reindex(GRUPOS_EDAD).dropna()
```

`groupby` ordena alfabéticamente. Con `reindex` se fuerza el orden natural de los tramos de edad; los tramos sin datos aparecen como `NaN` y se eliminan con `dropna()`.

### 5.5 `isin` y `duplicated`

- `df["col"].isin(conjunto)` devuelve True si el valor está en el conjunto. Con `~` delante, True si **no** está (valor no permitido).
- `df["col"].duplicated(keep="first")` marca como True las repeticiones, pero no la primera aparición.

### 5.6 Muestra reproducible: `sample`

```
completo.sample(n=5000, random_state=42).sort_index()
```

- `random_state` fija la **semilla**: la muestra es siempre la misma, lo que hace que los resultados (y el test de integración) sean reproducibles.
- `sort_index()` recupera el orden original de las filas.
- Al leer con `dtype=str` y `keep_default_na=False`, los valores se copian **exactamente** como vienen (`?`, `None`...).

### 5.7 `isinstance`

`isinstance(objeto, Clase)` comprueba si un objeto es de una clase. Se usa en los tests para verificar que `crear_repositorio` devuelve el repositorio correcto.

---

## 6. Datos grandes, licencias y Git

- **No se suben ficheros grandes al repo:** el completo (18 MB) va en `.gitignore`, y se sube una **muestra** más un **script documentado** para regenerarla.
- **Licencia CC BY 4.0:** permite usar y compartir los datos **citando la fuente**. Por eso el README incluye la URL, el DOI, la cita, la licencia y la fecha de descarga.
- `.gitignore` para un fichero concreto: `1_programacion_avanzada_python/Datos/diabetic_data.csv`.

---

## 7. Tests ( `tests/test_diabetes.py` )

| Tipo | Tests |
|---|---|
| Fixture en memoria | `encuentros_mini` (4 encuentros: 2 válidos, 1 fallecido, 1 sexo inválido) y la función auxiliar `crear_encuentro(**cambios)` |
| Validación | Recuento de válidos/inválidos, columna `motivo`, `parametrize` con los límites de `time_in_hospital` (0, 1, 14, 15), `readmitted` no permitido, diagnóstico vacío, duplicados y `ValidationError` por falta de columnas |
| Carga | `DataLoadError` si no existe el fichero; `?` → nulo y `None` → texto |
| Métricas | La distribución suma 100; pacientes recurrentes; estancia media |
| Integración | La muestra real da 4 902 válidos y 98 inválidos |

**`**cambios`** (argumentos con nombre variables):** `crear_encuentro(gender="Unknown/Invalid")` crea un encuentro válido y cambia solo ese campo. Los `**` recogen todos los argumentos con nombre en un diccionario.

**Límites en `parametrize`:** con una regla de rango [1, 14] se prueban los dos bordes (1 y 14, válidos) y justo fuera de ellos (0 y 15, inválidos).

---

## 8. Siguientes pasos para el proyecto de ML (fuera del Proyecto I)

El CSV limpio es la entrada del modelo. Pasos habituales en este dataset:

1. Eliminar `weight` (97 % nulos) y valorar qué hacer con `payer_code` y `medical_specialty`.
2. **Evitar fugas de información entre pacientes:** quedarse con un encuentro por paciente, o separar train y test **por paciente** ( `patient_nbr` ), no por fila.
3. Agrupar los diagnósticos (códigos ICD-9 de `diag_1` a `diag_3`) en categorías.
4. Definir el objetivo binario, por ejemplo `readmitted == "<30"`, y tratar el desbalance (≈ 11 % positivos).

---

## 9. Preguntas típicas de examen

1. **¿Por qué se usa `keep_default_na=False` en este dataset?** Porque pandas convertiría el texto `"None"` en `NaN`, y aquí `None` significa "prueba no realizada", que es un valor válido.
2. **¿Por qué se excluyen los pacientes fallecidos?** Porque no pueden reingresar; incluirlos sesgaría las tasas de reingreso.
3. **¿Por qué `weight` no invalida filas si tiene un 97 % de nulos?** Porque se perdería casi todo el dataset por una columna que no se usa en el análisis; se informa como columna casi vacía.

4. **¿Cómo se calcula una tasa por grupo en pandas?** Con la media de una máscara booleana agrupada: `(df["y"] == valor).groupby(df["grupo"]).mean()`.
5. **¿Cómo se cumple OCP en el validador?** Las reglas son datos (diccionarios y listas) que se recorren en bucles; para añadir una regla basta con añadir una entrada.
6. **¿Por qué se sube una muestra y no el dataset completo?** Para no subir ficheros grandes a Git. La muestra más el script de descarga permiten reproducir el trabajo.
7. **¿Para qué sirve `random_state` en `sample`?** Para que la muestra sea siempre la misma y los resultados sean reproducibles.
8. **¿Qué se reutiliza de la Parte A y qué se adapta?** Se reutilizan la arquitectura, el patrón Repository, las excepciones, el CLI y el informe de calidad. Se adaptan las reglas, las métricas, la lectura de nulos y los nombres del dominio.
9. **¿Qué obliga a cumplir la licencia CC BY 4.0?** A citar la fuente (autores, URL, licencia) al usar o compartir los datos.